Agentic Self-Healing in AI Software Engineering

A Study of *Reflexion: Language Agents with Verbal Reinforcement Learning*

CAQ413 Seminar

Seminar Report submitted in partial fulfillment for the degree
B.Tech in Artificial Intelligence and Data Science

Submitted by

Jino Jenz (Reg No: MUT23CA035)



Department of Artificial Intelligence and Data Science
(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that the seminar report entitled **"Agentic Self-Healing in AI Software Engineering"**, submitted by **Jino Jenz (MUT23CA035)** of Semester VII B.Tech Artificial Intelligence and Data Science, is a bonafide record of the work done under my supervision during the academic year 2026–27.

Guide	**Coordinators**	**Head of the Department**
Ms. Nimmi M K	Ms. Thasni K N & Ms. Vishnupriya Vijay	Dr. Cerene Mariam Abraham
Asst. Professor	Asst. Professors	Associate Professor
Dept. of AI & DS	Dept. of AI & DS	Dept. of AI & DS

ACKNOWLEDGEMENT

I am grateful to Dr. Neelakantan P. C., Principal, Muthoot Institute of Technology and Science, for providing the facilities and the academic environment in which this seminar was prepared.

I thank Dr. Cerene Mariam Abraham, Head of the Department of Artificial Intelligence and Data Science, for her support and for the opportunity to take up this topic.

I owe a special word of thanks to my seminar guide, Ms. Nimmi M K, Assistant Professor, Department of AI & DS. Her comments on the early drafts, and her questions about what the base paper actually claims, shaped the direction of this report.

I also thank the seminar coordinators, Ms. Thasni K N and Ms. Vishnupriya Vijay, Assistant Professors, Department of AI & DS, for reviewing the work at each stage and pointing out what needed to be corrected.

I thank the management of Muthoot Institute of Technology and Science, and all the teaching and non-teaching staff of the department, for their help during the semester. Finally, I thank my classmates of S7 AI&DS and my family for their support throughout.

Jino Jenz
(Reg. No: MUT23CA035)

Abstract

Modern software systems are large and tightly connected, so finding, diagnosing and repairing a failure often takes more time than writing the original code. Agentic AI offers a different approach: an agent built on a large language model (LLM) observes the outcome of its own work, decides why it failed, and tries again with a corrected plan. This seminar studies that closed loop through the Reflexion framework proposed by Shinn et al. (NeurIPS 2023). Reflexion improves an agent without changing any model weights. After each failed attempt, an Evaluator produces a pass/fail or scalar signal, a Self-Reflection model turns that signal into a short written diagnosis, and the diagnosis is stored in an episodic memory that the Actor reads on its next attempt. The report reviews the earlier work on reasoning agents and automated program repair that led to this design, explains the Actor–Evaluator–Self-Reflection architecture and its algorithm, and examines the reported results. With self-generated unit tests, Reflexion reaches 91.0% pass@1 on HumanEval (against 80.1% for GPT-4), solves 130 of 134 ALFWorld tasks, and improves HotPotQA accuracy by about 20 percentage points over a ReAct baseline. The report also discusses where the method fails: weak self-written tests on MBPP, smaller models, and tasks that need wide exploration. These findings show both what verbal feedback can contribute to self-healing software and where it still needs support.

Keywords: Agentic AI, Self-Healing Software, Reflexion, Verbal Reinforcement Learning, Large Language Models, Episodic Memory, Automated Code Repair, Automated Testing.

Contents

Contents i

List of Figures iii

List of Tables iv

1 Introduction v
- 1.1 Relevance to Self-Healing Software . . . vi
- 1.2 Objectives of the Seminar . . . vi
- 1.3 Organisation of the Report . . . vi

2 Literature Review vii
- 2.1 Chain-of-Thought Prompting (Wei et al., 2022) . . . vii
- 2.2 ReAct: Reasoning and Acting (Yao et al., 2023) . . . vii
- 2.3 Self-Refine (Madaan et al., 2023) . . . viii
- 2.4 Toolformer (Schick et al., 2023) . . . viii
- 2.5 AlphaCode and CodeT . . . viii
- 2.6 Self-Debugging (Chen et al., 2024) . . . viii
- 2.7 CodeRL (Le et al., 2022) . . . ix
- 2.8 Summary and Research Gap . . . ix

3 Datasets and Evaluation Setup xi
- 3.1 Benchmarks . . . xi
- 3.2 Experimental Setup . . . xii
- 3.3 Evaluation Metrics . . . xii

4 The Reflexion Framework xiv
- 4.1 Design Idea . . . xiv
- 4.2 Components . . . xiv
- 4.3 Formal Description . . . xv
- 4.4 Algorithm . . . xv
- 4.5 System Flow . . . xvi
- 4.6 Worked Example . . . xvii

5 Results and Discussion xviii
- 5.1 Decision-Making: ALFWorld . . . xviii
- 5.2 Reasoning: HotPotQA . . . xix
- 5.3 Programming . . . xx
- 5.4 Analysis of Test Outcomes . . . xx

5.5 Ablation Study xxi
5.6 Effect of Model Size xxii
5.7 Discussion xxii
5.8 Limitations of this Review xxii

6 Advantages **xxiv**

7 Disadvantages **xxv**

8 Applications **xxvii**

9 Conclusion **xxviii**

References **xxix**

A Presentation Slides **xxxi**

B Base Paper **xxxii**

C Viva Questions and Answers **xxxiii**

D Similarity Index Report **xxxv**

E Alignment with Sustainable Development Goals (SDG) **xxxvi**

List of Figures

4.1 Flow of data and feedback in one Reflexion trial xvi

5.1 ALFWorld success rate over trials (reproduced from Shinn et al. [1]) xix
5.2 HotPotQA success rate over trials (reproduced from Shinn et al. [1]) xx

B.1 First page of the base paper by Shinn et al. [1] xxxii

List of Tables

2.1 Comparison of related methods with Reflexion x

3.1 Categories used for analysing code results xiii

5.1 Pass@1 accuracy on code benchmarks (values from [1]) xx
5.2 Accuracy and test-outcome rates for HumanEval and MBPP (values from [1]) xxi
5.3 Ablation on the 50 hardest HumanEval Rust problems (values from [1]) . . . xxi

Chapter 1

Introduction

Large language models are now used as the core of autonomous agents that act in an external environment: they call APIs, run code, browse the web and play text-based games. Prompting methods such as Chain-of-Thought (CoT) [2] and ReAct [3] let a single model break a task into steps and decide on an action at each step. What these agents cannot do well is learn from a failed attempt. A conventional reinforcement learning (RL) agent learns by trial and error, but it needs a very large number of episodes and gradient updates to its parameters [4]. For a model with billions of parameters, accessed through an API, that kind of training is usually not possible.

The simplest alternative is to retry. If an attempt fails, the agent is reset and runs again. A plain retry, however, keeps no record of what went wrong. The agent tends to repeat the same mistake, because nothing in its input has changed. In a long task, one wrong action early in the trajectory can lead to many more, and the agent has no way of seeing which step caused the failure.

Reflexion, proposed by Shinn et al. [1], addresses this gap. Instead of updating weights, the agent writes down, in plain language, what went wrong and what it should do differently. This note is stored in a small memory and placed in the prompt for the next attempt. The authors describe the note as a “semantic gradient”: it gives the agent a direction for improvement in the same way a gradient does in ordinary training, but it is expressed as text.

1.1 Relevance to Self-Healing Software

A self-healing software system is one that detects a fault, works out its cause, applies a fix and checks that the fix holds, with little or no human intervention. Reflexion follows the same cycle at the level of a single agent. The Evaluator detects the failure, the Self-Reflection model diagnoses it, the Actor applies a new attempt, and the Evaluator checks the result again. The programming experiments in the paper are the clearest example: the agent writes a function, writes its own unit tests, runs them, and uses the failing tests to repair the code. This makes Reflexion a useful case study for agentic self-healing in software engineering.

1.2 Objectives of the Seminar

- To review the earlier work on reasoning agents and automated program repair that Reflexion builds on.
- To explain the Actor, Evaluator and Self-Reflection components and the algorithm that connects them.
- To examine the results reported on ALFWorld, HotPotQA, HumanEval, MBPP and LeetcodeHardGym, including the cases where Reflexion does worse than the baseline.
- To identify the strengths, limitations and possible applications of the approach for self-healing software.

1.3 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 describes the benchmarks and evaluation measures used in the base paper. Chapter 4 explains the Reflexion framework in detail, and Chapter 5 presents and discusses the reported results. Chapters 6 to 8 cover advantages, disadvantages and applications, and Chapter 9 concludes the report.

Chapter 2

Literature Review

This chapter covers the main lines of work that Reflexion draws on. The first group of papers deals with reasoning and acting in language models. The second deals with code generation where the model receives feedback from tests or an interpreter.

2.1 Chain-of-Thought Prompting (Wei et al., 2022)

Wei et al. [2] showed that adding worked, step-by-step examples to a prompt makes large models much better at arithmetic, commonsense and symbolic reasoning. The model writes its intermediate reasoning before the final answer. CoT is a single forward pass, however. If the reasoning goes wrong, the model has no signal from outside to tell it so, and running it again gives a similar answer. Reflexion uses CoT as one of its Actor variants for the HotPotQA experiments.

2.2 ReAct: Reasoning and Acting (Yao et al., 2023)

ReAct [3] interleaves free-text “thoughts” with actions such as a Wikipedia search or a move in a text game. The observation returned by each action is added to the context, so the model can adjust its plan within one episode. ReAct performed well on HotPotQA and ALFWorld. Its limitation is that learning stops at the end of the episode. If the agent becomes convinced early on that it is holding an object it never picked up, it continues with

that false belief until the step limit is reached. Reflexion uses ReAct as the Actor for its decision-making experiments and as the main baseline.

2.3 Self-Refine (Madaan et al., 2023)

Self-Refine [5] asks the same model to critique its own output and then rewrite it, repeating this a few times. It improved results on tasks such as code optimisation and dialogue without any fine-tuning. The method works on a single generation, though, and it does not keep a memory that carries over between separate attempts at a task. It also relies only on the model's own opinion of its output, with no signal from an environment.

2.4 Toolformer (Schick et al., 2023)

Toolformer [6] trains a model to decide when to call external tools such as a calculator or a search engine, and how to use the results. It shows that external tools can correct gaps in a model's knowledge. However, learning when to use a tool requires fine-tuning, which is exactly the cost Reflexion tries to avoid.

2.5 AlphaCode and CodeT

AlphaCode [7] generates a very large number of candidate programs and filters them using the example tests given in the problem. CodeT [8] has the model write its own unit tests and then ranks candidate programs by how many of these tests they pass. CodeT does not need hidden test cases, which keeps its pass@1 results fair. Both methods choose among existing candidates; neither uses a failed test to improve the next program.

2.6 Self-Debugging (Chen et al., 2024)

Self-Debugging [9] feeds execution results and error messages back to the model and asks it to correct the code, sometimes after explaining the code line by line. This works well for

syntax errors and small logic bugs. In its main setting it uses ground-truth tests to decide when the code is correct, and it does not keep a summary of earlier failures that the model can consult later.

2.7 CodeRL (Le et al., 2022)

CodeRL [10] treats code generation as a reinforcement learning problem. A pretrained model acts as the actor and a separate critic network predicts whether a program will pass its unit tests; the critic's score guides fine-tuning. The approach is principled but expensive, because it needs gradient updates to a large model. A single scalar reward also makes it hard to tell which part of a long program caused the failure.

2.8 Summary and Research Gap

Table 2.1 compares these methods on the properties that matter for a self-correcting agent.

The gap that Reflexion fills is the combination in the last row: feedback from the environment or from tests, converted into a written explanation, kept in memory across attempts, and used without any training of the model.

Table 2.1: Comparison of related methods with Reflexion

Method	**Feedback used**	**Memory across attempts**	**Weight updates**	**Main limitation**
CoT [2]	None	No	No	No signal from outside the model
ReAct [3]	Environment observations	No	No	Learning ends with the episode
Self-Refine [5]	Model's own critique	No	No	Single-generation tasks only
CodeT [8]	Self-written tests	No	No	Ranks candidates, does not repair
Self-Debugging [9]	Interpreter errors	No	No	No stored reflection
CodeRL [10]	Critic reward	Implicit (weights)	Yes	High training cost
Reflexion [1]	Any of the above, turned into text	Yes	No	Depends on evaluator quality

Chapter 3

Datasets and Evaluation Setup

This chapter describes the benchmarks, the preparation steps and the measures that Shinn et al. [1] used to evaluate Reflexion. All experiments discussed in this report are those of the original authors; no new experiments were run for this seminar.

3.1 Benchmarks

The authors chose three kinds of task so that the method would be tested on long action sequences, on reasoning over text, and on writing code.

- **ALFWorld (sequential decision-making):** ALFWorld [11] is a set of text-based household environments. The agent must, for example, find a spatula in a drawer or chill a tomato in the fridge, using text commands. The paper uses 134 environments across six task types.
- **HotPotQA (multi-hop reasoning):** HotPotQA [12] contains questions that can only be answered by combining facts from two or more Wikipedia articles. The paper evaluates on a sample of 100 questions, using exact match against the reference answer.
- **Code generation (HumanEval, MBPP and LeetcodeHardGym):** HumanEval [13] has 164 Python function-writing problems and MBPP [14] contains short Python tasks of basic difficulty. Both were also translated to Rust using MultiPL-E [15].

The authors added a new set, LeetcodeHardGym, of 40 hard Leetcode problems released after GPT-4's training cut-off, so that the model could not have seen the solutions during pre-training.

3.2 Experimental Setup

- **Few-shot prompts:** For ALFWorld, the agent was given two example trajectories, the same ones used in the ReAct paper. HotPotQA used six examples for CoT, two for ReAct and two for the reflection step.
- **Self-generated unit tests:** For the coding tasks, the agent wrote its own tests. Each proposed test was parsed into an abstract syntax tree (AST) and discarded if it was not valid code. At most six tests were then kept to form the internal test suite. This means the agent never sees the hidden tests, so the results count as true pass@1.
- **Memory limit:** To keep prompts within the model's context length, only the most recent reflections are kept. The limit Ω is three reflections for ALFWorld and HotPotQA and one for the programming tasks.
- **Stopping rule for ALFWorld:** A hand-written heuristic decides when an attempt has failed: the same action giving the same response more than three times, or more than 30 actions in one environment.

3.3 Evaluation Metrics

- **Pass@1 accuracy:** The share of problems for which the single program finally submitted passes the hidden tests.
- **Success rate over trials:** For ALFWorld and HotPotQA, the share of tasks solved after each trial. This shows whether the agent keeps improving or levels off.
- **Test-outcome categories:** For code, each final result is placed in one of four groups, defined in Table 3.1. The false positive rate is the most important of these, because

it measures how often the agent's own tests let a wrong program through.

- **Ablation accuracy:** Pass@1 when one part of the system (test generation or reflection) is removed. This shows how much each part contributes.

Table 3.1: Categories used for analysing code results

Category	**Internal tests**	**Hidden tests**
True Positive (TP)	Pass	Pass
False Negative (FN)	Fail	Pass
False Positive (FP)	Pass	Fail
True Negative (TN)	Fail	Fail

Chapter 4

The Reflexion Framework

This chapter explains how Reflexion is put together: the three models, the two kinds of memory, and the loop that connects them.

4.1 Design Idea

In ordinary RL, a reward signal is turned into a change in the network's weights. Reflexion keeps the weights fixed and changes the input instead. The "policy" of the agent is defined as the pair of the language model and the contents of its memory. Improving the policy therefore means improving what is written in memory. Because that memory is plain text, a developer can read exactly what the agent has learned after each attempt.

4.2 Components

- **Actor (M_a):** An LLM prompted to produce actions or text based on the current state. In the experiments it is either a CoT or a ReAct agent. It reads two kinds of memory: the current trajectory (short-term) and the stored reflections (long-term).
- **Evaluator (M_e):** Scores a finished trajectory. The scoring method depends on the task: exact match for HotPotQA, the stopping heuristic or an LLM classifier for ALF-World, and the self-written unit tests for programming.

- **Self-Reflection model (M_{sr}):** A second LLM call that receives the failed trajectory, its score and the existing memory, and writes a short, first-person explanation of what went wrong and what to do next time.

4.3 Formal Description

Let π_θ be the agent's policy, where $\theta = \{M_a, mem\}$. In trial t the Actor produces a trajectory

$$\tau_t = [a_0, o_0, a_1, o_1, \ldots, a_i, o_i] \sim \pi_\theta, \tag{4.1}$$

where a_j are actions and o_j are observations from the environment. The Evaluator gives a scalar reward

$$r_t = M_e(\tau_t), \tag{4.2}$$

and the Self-Reflection model turns the trajectory and reward into a written reflection

$$sr_t = M_{sr}(\tau_t, r_t, mem). \tag{4.3}$$

The reflection is appended to memory, and only the last Ω entries are kept:

$$mem \leftarrow \text{last}_\Omega\left(mem \cup \{sr_t\}\right), \qquad \Omega \in \{1, 2, 3\}. \tag{4.4}$$

4.4 Algorithm

Algorithm 1 gives the complete loop. It is adapted from Algorithm 1 of the base paper [1]; the loop condition has been written so that it stops as soon as the Evaluator accepts the trajectory or the trial limit is reached.

Algorithm 1 Reinforcement through self-reflection

Require: Actor M_a, Evaluator M_e, Self-Reflection model M_{sr}, trial limit T, memory size Ω

$mem \leftarrow [\,], \quad t \leftarrow 0$
Generate trajectory τ_0 with π_θ
$r_0 \leftarrow M_e(\tau_0)$
while r_t is not a pass **and** $t < T$ **do**
    $sr_t \leftarrow M_{sr}(\tau_t, r_t, mem)$
    Append sr_t to mem and keep only the last Ω entries
    Reset the environment; $t \leftarrow t + 1$
    Generate τ_t with π_θ, conditioned on mem
    $r_t \leftarrow M_e(\tau_t)$
end while
return τ_t

4.5 System Flow

Figure 4.1 shows how information moves between the parts of the system during one trial.

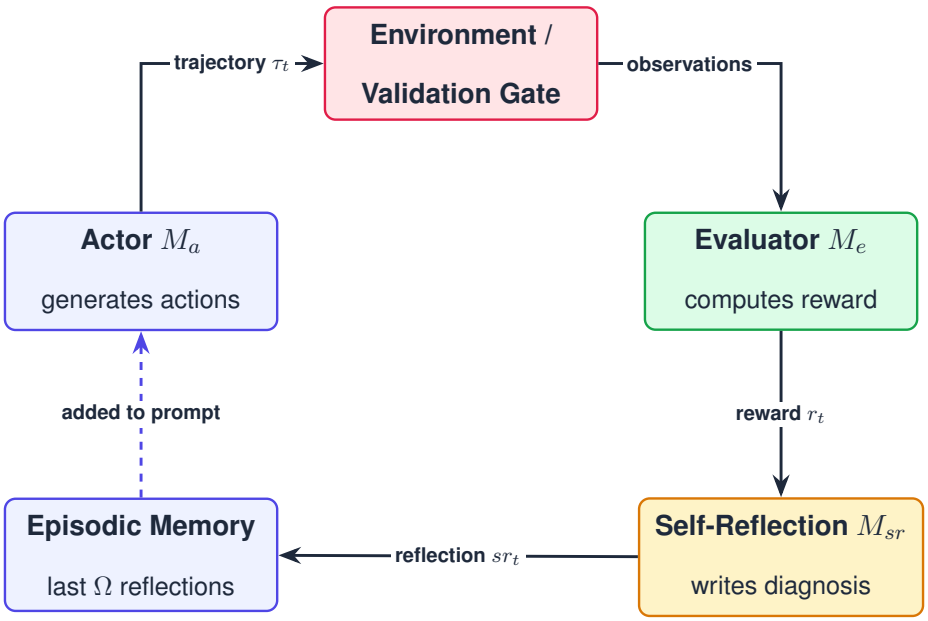


Figure 4.1: Flow of data and feedback in one Reflexion trial

The Actor acts in the environment and produces a trajectory. The Evaluator scores it. If

the attempt failed, the Self-Reflection model writes a diagnosis, which is stored in memory. On the next trial the Actor reads this memory together with the task, so its plan changes before it takes the first action.

4.6 Worked Example

The appendix of the base paper [1] includes an ALFWorld episode with the task “examine the mug with the desklamp”. In the first trial, the agent picks up the mug first and then searches for the lamp. It reaches the step limit without completing the task. The reflection it writes after this failure is:

```
Reflection: In this environment, my plan was to find a mug then find and use a
    desklamp. However, the task says to examine the mug with the desklamp. I should
    have looked for the desklamp first, then looked for the mug. I noticed that the
    desklamp was found on desk 1. In the next trial, I will go to desk 1, find the
    lamp, then look for the mug and examine it with the desklamp.
```

In the second trial the agent goes straight to desk 1, takes the mug that is also there, switches on the lamp and finishes the task in a few steps. The reflection names the wrong decision (the order of the two sub-goals) and records a useful fact (where the lamp is). A plain retry would have kept neither, which is why it often fails in the same way again.

Chapter 5

Results and Discussion

The results in this chapter are those reported by Shinn et al. [1]. The figures are reproduced from the plots in the base paper.

5.1 Decision-Making: ALFWorld

Figure 5.1 shows the share of the 134 ALFWorld environments solved after each trial. All three agents start at about 0.63. The ReAct-only agent improves for a few trials and then stops at about 0.75 around trials 6–7. With Reflexion, the agent keeps improving and reaches about 0.97 using the heuristic evaluator, solving 130 of the 134 tasks. Using an LLM as the evaluator gives a similar curve, slightly lower.

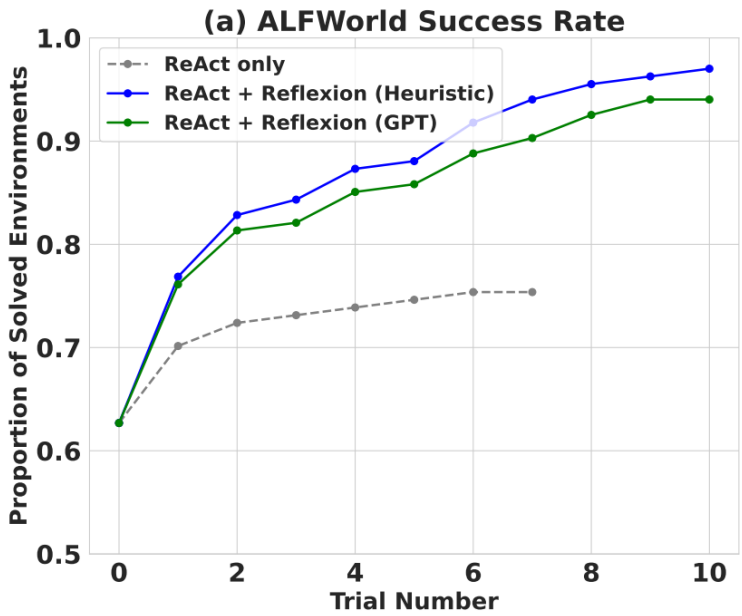


Figure 5.1: ALFWorld success rate over trials (reproduced from Shinn et al. [1])

The authors also classified the failures. The most common one in the baseline was hallucination: the agent believed it was holding an item that it did not have. The ReAct-only agent remained at a hallucination rate of about 22% with no sign of recovery, while Reflexion removed almost all of these cases.

5.2 Reasoning: HotPotQA

Figure 5.2 shows results on 100 HotPotQA questions. The CoT-only and ReAct-only agents stay at about 0.32–0.34 over all trials. None of the questions they fail in the first trial is solved later, even at a sampling temperature of 0.7. ReAct with Reflexion rises to 0.54 by the sixth trial, and CoT with Reflexion rises from 0.32 to 0.40.

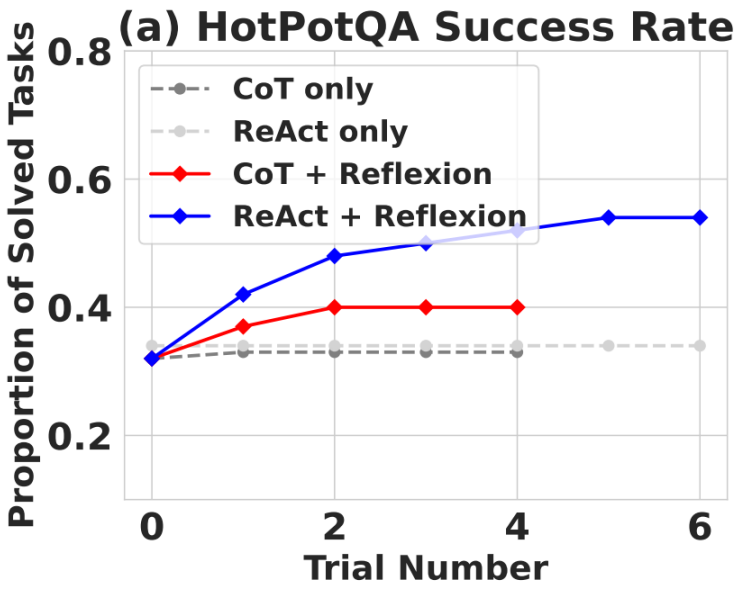


Figure 5.2: HotPotQA success rate over trials (reproduced from Shinn et al. [1])

An ablation in the same section separates the effect of memory from the effect of reflection. Giving the agent its last trajectory as memory helps, but adding the written reflection on top of it gives a further 8 percentage point gain. Storing what happened is useful; storing an explanation of why it failed is more useful.

5.3 Programming

Table 5.1 compares pass@1 accuracy with the previous best results. Reflexion improves on GPT-4 in every setting except MBPP Python.

Table 5.1: Pass@1 accuracy on code benchmarks (values from [1])

Benchmark + Language	Earlier best	GPT-4	Reflexion
HumanEval (Python)	65.8 (CodeT + GPT-3.5)	80.1	**91.0**
HumanEval (Rust)	–	60.0	**68.0**
MBPP (Python)	67.7 (CodeT + Codex)	**80.1**	77.1
MBPP (Rust)	–	70.9	**75.4**
LeetcodeHard (Python)	–	7.5	**15.0**

5.4 Analysis of Test Outcomes

Table 5.2 breaks the code results into the four categories defined in Table 3.1.

Table 5.2: Accuracy and test-outcome rates for HumanEval and MBPP (values from [1])

Benchmark + Language	Base	Reflexion	TP	FN	FP	TN
HumanEval (Python)	0.80	**0.91**	0.99	0.40	0.01	0.60
MBPP (Python)	**0.80**	0.77	0.84	0.59	0.16	0.41
HumanEval (Rust)	0.60	**0.68**	0.87	0.37	0.13	0.63
MBPP (Rust)	0.71	**0.75**	0.84	0.51	0.16	0.49

Why Reflexion is worse on MBPP Python

MBPP Python is the one case where Reflexion scores lower than the base model (77.1% against 80.1%). The baseline accuracy on HumanEval Python and MBPP Python is similar, so the difference does not come from the problems being harder. It comes from the agent's own tests. On MBPP Python, 16.3% of the programs that passed all internal tests were actually wrong. On HumanEval Python this rate was only 1.4%.

A false positive is the more harmful error for Reflexion. When the internal tests pass, the loop stops and the wrong program is submitted, so the agent never gets a chance to reflect on it. A false negative is less damaging: a correct program fails a badly written test, and the reflection step can sometimes recognise that the test is wrong and keep the code. The MBPP problem descriptions are short and sometimes ambiguous, which makes it harder to write tests that check the intended behaviour [16]. The weakness, therefore, lies in the Evaluator, not in the reflection step.

5.5 Ablation Study

The authors removed one part at a time on the 50 hardest HumanEval Rust problems, with GPT-4 as the base model. Table 5.3 shows the result.

Table 5.3: Ablation on the 50 hardest HumanEval Rust problems (values from [1])

Approach	Test generation	Self-reflection	Pass@1
Base model	No	No	0.60
Without test generation	No	Yes	0.52
Without self-reflection	Yes	No	0.60
Full Reflexion	Yes	Yes	**0.68**

Without tests, accuracy falls below the baseline to 0.52. The agent cannot tell whether its code is already correct, so it keeps editing and sometimes breaks working code. Without the reflection step, the tests still catch the errors, but the fixes do not address them and accuracy stays at 0.60. Only with both parts does accuracy rise. The two components depend on each other: tests detect the fault, and reflection turns that detection into a useful repair.

5.6 Effect of Model Size

The appendix of the paper repeats the HumanEval Python experiment with starchat-beta, a much smaller open model. Its pass@1 accuracy is 0.26 with and without Reflexion. On HotPotQA, the gain from Reflexion is present for text-davinci-003, gpt-3.5-turbo and GPT-4, but it varies a great deal between them. The ability to write a useful reflection appears only in large models.

5.7 Discussion

Three points stand out from these results. First, the largest gains come where the Evaluator is reliable: the ALFWorld heuristic, exact match in HotPotQA, and HumanEval Python with a 1.4% false positive rate. Second, a written explanation adds value beyond simple memory, as both the HotPotQA and the Rust ablations show. Third, Reflexion makes the agent's learning visible. Each reflection is a short note that explains a change in behaviour, which is useful for anyone who needs to check why a self-healing system made a particular fix.

5.8 Limitations of this Review

This report is based on the published results and was not able to reproduce the experiments. The plots are taken from the paper. The original experiments used GPT-3, GPT-3.5 and GPT-4 versions available in 2023, so the exact numbers may differ with newer models.

HotPotQA was evaluated on only 100 questions, which is a small sample for drawing firm conclusions.

Chapter 6

Advantages

- **No training required:** All improvement happens through the prompt. There is no backpropagation and no fine-tuning, so the method works with models that are only available through an API.
- **Readable learning:** Every change in the agent's behaviour comes with a written reason. A developer can read the memory and see what the agent concluded after each failure, which is not possible with a weight update.
- **Better credit assignment than a scalar reward:** A single number does not say which step in a long trajectory caused the failure. A reflection can point to the exact action, as in the desklamp example in Chapter 4.
- **Works with many kinds of feedback:** The Evaluator can be a heuristic, an exact-match check, an LLM judge or a set of unit tests. The rest of the system stays the same.
- **Strong reported results:** 91.0% pass@1 on HumanEval Python, 130 of 134 ALF-World tasks solved, and a gain of about 20 percentage points on HotPotQA over ReAct.
- **Modular design:** The Actor, Evaluator and Self-Reflection model are separate parts. Each can be replaced, for example by a stronger test generator, without redesigning the loop.

Chapter 7

Disadvantages

- **Only as good as the Evaluator:** If the agent's own tests are weak, wrong programs pass and the loop stops early. On MBPP Python this happened for 16.3% of passing programs, and Reflexion scored below the baseline.
- **Needs a large model:** With a small model such as starchat-beta, reflection gave no improvement. This limits use on local or low-cost hardware.
- **Weak at open-ended exploration:** On WebShop [17], an online-shopping task, the agent showed no improvement after four trials and its reflections were not useful. The method can settle on a poor solution when escaping it needs a very different approach.
- **No convergence guarantee:** Unlike some RL methods, there is no proof that the loop will reach a correct solution. It can repeat similar reflections without making progress.
- **Context and cost:** Every reflection is added to the prompt. The memory has to be kept small (Ω = 1–3), and each trial adds more API calls, so cost and latency grow with the number of trials.
- **Safety of running generated code:** The paper advises running the code-writing agent only in an isolated environment, since generated code is executed before it is

checked. Code that uses network APIs, randomness or concurrency is also hard to test reliably.

Chapter 8

Applications

- **Self-healing software pipelines:** When a build or a test fails in a continuous integration system, an agent can read the error, write down the likely cause, propose a patch and run the tests again. The stored reflections act as a record of what was tried.
- **Coding assistants in IDEs:** An assistant that writes code can also write tests, run them in a sandbox and fix its own output before showing it to the developer, reducing the number of broken suggestions.
- **Teaching programming:** The reflections explain an error in plain language, for example “the function only checks that the counts of brackets are equal, not their order”. Students can learn more from this than from a stack trace.
- **Question answering and research tools:** A search agent can note which queries failed and why, and change its search strategy on the next attempt, as in the HotPotQA experiments.
- **Testing and benchmarking LLMs:** Benchmarks such as LeetcodeHardGym, together with the stored reflections, show where current models make repeated mistakes, which helps in building better test sets and prompts.

Chapter 9

Conclusion

This seminar examined Reflexion as an example of agentic self-healing. The framework improves a language agent by having it write and remember an explanation of each failure, instead of updating the model's weights. Its three parts, the Actor, the Evaluator and the Self-Reflection model, follow the same detect–diagnose–repair–verify cycle that a self-healing software system needs.

The reported results support the idea. Reflexion reached 91.0% pass@1 on HumanEval Python, solved 130 of 134 ALFWorld tasks and gained about 20 percentage points on HotPotQA. The ablation study showed that the gains come from tests and reflection working together; neither is enough alone.

The weaknesses are just as clear. The method is limited by the quality of its Evaluator, as the false positives on MBPP Python show. It needs large models, and it does not help on tasks that require broad exploration, such as WebShop. For real self-healing systems, this means that the test or check used to judge a fix deserves as much attention as the model that writes it.

Future work could store reflections in a searchable database instead of a short sliding window, combine Reflexion with stronger test generation to reduce false positives, and test whether smaller, newer models can now produce useful reflections. Measuring how accurate the reflections themselves are, and not only the final task score, would also make the approach easier to trust in practice.

References

[1] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[5] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[7] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[8] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code generation with generated tests," in *International Conference on Learning Representations (ICLR)*, 2023.

[9] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.

[10] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 21 314–21 328.

[11] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, "ALFWorld: Aligning text and embodied environments for interactive learning," in *International Conference on Learning Representations (ICLR)*, 2021.

[12] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380.

[13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[14] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[15] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, A. Guha, M. Greenberg, and A. Jangda, "MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation," *IEEE Transactions on Software Engineering*, vol. 49, no. 7, pp. 3675–3691, 2023.

[16] W. Lam, S. Winter, A. Wei, T. Xie, D. Marinov, and J. Bell, "A large-scale longitudinal study of flaky tests," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–29, 2020.

[17] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "WebShop: Towards scalable real-world web interaction with grounded language agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 20 744–20 757.

Appendix A

Presentation Slides

Appendix B

Base Paper

arXiv:2303.11366v4 [cs.AI] 10 Oct 2023

Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn
Northeastern University
noahshinn024@gmail.com

Federico Cassano
Northeastern University
cassano.f@northeastern.edu

Edward Berman
Northeastern University
berman.ed@northeastern.edu

Ashwin Gopinath
Massachusetts Institute of Technology
agopi@mit.edu

Karthik Narasimhan
Princeton University
karthikn@princeton.edu

Shunyu Yao
Princeton University
shunyuy@princeton.edu

Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead through linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) and sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at https://github.com/noahshinn024/reflexion.

1 Introduction

Recent works such as ReAct [30], SayCan [1], Toolformer [22], HuggingGPT [23], generative agents [19], and WebGPT [17] have demonstrated the feasibility of autonomous decision-making agents that are built on top of a large language model (LLM) core. These methods use LLMs to generate text and 'actions' that can be used in API calls and executed in an environment. Since they rely on massive models with an enormous number of parameters, such approaches have been so far limited to using in-context examples as a way of teaching the agents, since more traditional optimization schemes like reinforcement learning with gradient descent require substantial amounts of compute and time.

Preprint. Under review.

Figure B.1: First page of the base paper by Shinn et al. [1]

Appendix C

Viva Questions and Answers

1. **What is Reflexion?**
   A framework in which a language agent improves over repeated attempts by writing a short explanation of each failure and reading it before the next attempt. The model's weights are never changed.

2. **Why is it called verbal reinforcement learning?**
   In RL, a reward is used to improve the policy. In Reflexion, the reward is converted into a written reflection, and the policy improves because its input (the memory) improves.

3. **What are the three components?**
   The Actor (M_a), which acts; the Evaluator (M_e), which scores the attempt; and the Self-Reflection model (M_{sr}), which writes the diagnosis.

4. **What is the difference between short-term and long-term memory here?**
   Short-term memory is the trajectory of the current trial. Long-term memory is the list of stored reflections from earlier trials, limited to the last Ω entries.

5. **Why is the memory limited to one to three reflections?**
   Every reflection is added to the prompt, and the model has a fixed context length. A small window also keeps the most recent and relevant lessons in focus.

6. **How does Reflexion stay eligible for pass@1 on code tasks?**
   It never uses the hidden tests. The agent writes its own unit tests, checks them for syntax with an AST parser, and uses at most six of them.

7. **Why does Reflexion do worse than GPT-4 on MBPP Python?**
   Its self-written tests were weak: 16.3% of programs that passed them were wrong. The loop stopped early and submitted those programs.

8. **Which is worse for Reflexion, a false positive or a false negative?**
   A false positive, because the loop ends and the wrong answer is submitted. With a false negative, the agent can still reflect and may notice that the test itself is wrong.

9. **What did the ablation study show?**
   Removing test generation dropped accuracy to 0.52, and removing reflection left it at the baseline of 0.60. Only the full system reached 0.68.

10. **How is Reflexion different from Self-Refine?**
    Self-Refine improves one output through self-critique within a single attempt. Reflexion uses an external or test-based signal and keeps a memory that carries over between separate attempts.

11. **Where does Reflexion fail?**
    On tasks that need a lot of exploration, such as WebShop, and with small models such as starchat-beta.

12. **How does this relate to self-healing software?**
    The loop matches the self-healing cycle: the Evaluator detects the fault, the reflection diagnoses it, the Actor applies a fix, and the Evaluator verifies it.

Appendix D

Similarity Index Report

Appendix E

Alignment with Sustainable Development Goals (SDG)

- **SDG 9 – Industry, Innovation and Infrastructure:** Software is a core part of modern infrastructure. Agents that can find and repair faults on their own help keep that infrastructure reliable and reduce the time systems are out of service.
- **SDG 4 – Quality Education:** Written reflections explain programming mistakes in plain language. Used in teaching tools, they can give students specific feedback on their code instead of only an error message.
- **SDG 8 – Decent Work and Economic Growth:** Automating repetitive debugging and testing lets developers spend more of their time on design and on problems that need human judgement.
- **SDG 12 – Responsible Consumption and Production:** Reflexion improves results without retraining large models. Avoiding repeated fine-tuning saves the large amount of computing power and energy that training requires.